DOCUMENTAÇÃO TÉCNICA EXECUTIVA

# Desafio Backend Motor Rent

Relatório Completo de Arquitetura, Funcionalidades, Microsserviços e Guia Operacional

### Sumário do Sistema

Este relatório técnico apresenta detalhadamente a engenharia, arquitetura e funcionamento do ecossistema construído para o desafio de gestão de frota de motos, motoristas entregadores, locações e gerenciamento de entregas. O sistema adota os princípios de **Domain-Driven Design (DDD)** e **Clean Architecture**, fundamentado na plataforma **.NET 8**, mensageria distribuída com **RabbitMQ (MassTransit)**, persistência relacional com **PostgreSQL 16** e orquestração de containers com **Docker Compose**.

**Plataforma & Framework**
.NET 8 LTS (C# 12) / ASP.NET Core

**Orquestração & Infra**
Docker Compose / Alpine Linux

**Banco & Mensageria**
PostgreSQL 16 / RabbitMQ 3

**Data da Emissão**
Novembro / 2026

# 1. Visão Geral da Solução e Arquitetura

A solução foi projetada sob o paradigma de **Domain-Driven Design (DDD)** e **Clean Architecture** (Arquitetura Limpa), estabelecendo limites arquiteturais claros, isolamento de responsabilidades e desacoplamento absoluto do núcleo de negócios em relação a frameworks, drivers e infraestrutura.

API REST **DesafioBackend.Api (Porta 5000)**

Ponto de entrada HTTP RESTful conforme contrato OpenAPI/Swagger. Provê autenticação JWT Bearer, tratamento global de exceções, upload de documentos e injeção de dependências.

WEB MVC **DesafioBackend.Web (Porta 5001)**

Portal Administrativo em ASP.NET Core MVC. Permite gestão visual de motos, entregadores, locações com simulação de taxas e controle em tempo real de pedidos disponíveis, em trânsito e concluídos.

MENSAGERIA **RabbitMQ + MassTransit (Porta 5672 / 15672)**

Broker orientado a eventos assíncronos. Processa eventos de domínio como `MotorcycleRegisteredEvent`, filtrando motos do ano 2024 para auditoria e notificações reativas.

PERSISTÊNCIA **PostgreSQL 16 (Porta 5433 / 5432)**

Banco relacional com transações atômicas ACID, índices únicos de negócio (CNPJ, CNH, Placa), migrações automáticas via EF Core e volume persistente para segurança dos dados.

## Estrutura em Camadas da Solução

| Camada / Projeto | Responsabilidade Primária | Principais Componentes e Padrões |
|---|---|---|
| **DesafioBackend.Domain** | Regras de negócio corporativas e invariantes do sistema. Não possui dependência de frameworks. | • Entidades: `Motorcycle`, `DeliveryDriver`, `Rental`, `Delivery`, `MotorcycleNotification`<br>• Enums: `DriverLicenseType` (A, B, AB), `RentalPlanType` (7, 15, 30, 45, 50 dias), `DeliveryStatus`<br>• Eventos: `MotorcycleRegisteredEvent`<br>• Interfaces: `IMotorcycleRepository`, `IRentalRepository`, `IUnitOfWork` |
| **DesafioBackend.Application** | Orquestração dos casos de uso, contratos de entrada e saída (DTOs) e fluxos de aplicação. | • Serviços: `MotorcycleService`, `DeliveryDriverService`, `RentalService`, `DeliveryService`<br>• DTOs: Requests e Responses padronizados para o Swagger<br>• Interfaces de Infraestrutura: `IMessageBus`, `IStorageService`<br>• Exceções de Domínio: `ValidationException`, `NotFoundException`, `ConflictException` |
| **DesafioBackend.Infrastructure** | Implementação de acesso a dados, integração com mensageria e sistema de arquivos. | • EF Core 8 com Npgsql e Mapeamentos Fluent API (`AppDbContext`)<br>• Repositórios concretos e Unit of Work atômico<br>• MassTransit: `MassTransitMessageBus` e Consumer `MotorcycleCreatedConsumer`<br>• Storage: `LocalStorageService` com persistência em disco de arquivos PNG/BMP |
| **DesafioBackend.Api** | Interface de comunicação externa HTTP (REST API). | • Controllers: `MotosController`, `EntregadoresController`, `LocacaoController`, `EntregasController`, `AuthController`<br>• Middlewares: `GlobalExceptionMiddleware` (padronização de payload de erro)<br>• Swagger com configuração de autenticação JWT Bearer |
| **DesafioBackend.Web** | Interface Gráfica Web para Gestão Operacional e Administrativa. | • ASP.NET Core MVC com Razor Views e Bootstrap 5<br>• `ApiClient` com `JwtTokenHandler` para repasse transparente de credenciais<br>• Telas de Auditoria de Entregas Disponíveis, Em Andamento e Concluídas |

# 2. Infraestrutura e Orquestração com Docker Compose

Toda a stack de microsserviços e dependências é orquestrada através do arquivo `docker-compose.yml`, permitindo que a aplicação seja inicializada do zero com um único comando, sem necessidade de configuração prévia de bancos de dados ou servidores na máquina hospedeira.

## Topologia de Rede e Portas dos Serviços

| Serviço | Container Name | Porta Externa (Host) | Porta Interna (Docker) | Healthcheck & Dependência |
|---|---|---|---|---|
| **PostgreSQL 16** | `desafio_backend_postgres` | **5433** | 5432 | `pg_isready -U postgres -d desafiodb` |
| **RabbitMQ 3** | `desafio_backend_rabbitmq` | **5672** (AMQP)<br>**15672** (Web) | 5672<br>15672 | `rabbitmq-diagnostics -q ping` |
| **API Backend (.NET 8)** | `desafio_backend_api` | **5000** | 8080 | Inicia apenas após `db` e `rabbitmq` estarem *healthy* |
| **Painel Web MVC (.NET 8)** | `desafio_backend_web` | **5001** | 8080 | Inicia após a `api` estar operacional |

> 💡 **Migrações Automáticas do Banco de Dados:** Na inicialização do container `desafio_backend_api`, o método `context.Database.MigrateAsync()` é executado automaticamente no `Program.cs`. Desta forma, todas as tabelas, índices e restrições são criados sem requerer comandos manuais.

## Configuração do Dockerfile Multi-Stage

Tanto a API quanto o projeto Web utilizam **Dockerfiles multi-stage** baseados na imagem oficial `mcr.microsoft.com/dotnet/sdk:8.0` para compilação e `mcr.microsoft.com/dotnet/aspnet:8.0-alpine` para execução em produção. Isso garante:

- Tamanho reduzido de imagem (menos de 220 MB por container).
- Superfície de ataque mínima por utilizar o kernel Alpine Linux endurecido.
- Separação total entre ferramentas de build (SDK, compiladores) e o runtime final.

# 3. Mensageria Assíncrona e Arquitetura Orientada a Eventos (RabbitMQ)

O sistema adota mensageria para desacoplar operações de cadastro de frotas e processamento reativo. A comunicação com o broker RabbitMQ é abstraída através do **MassTransit**, framework padrão da indústria que provê resiliência, reconnect automático e tópicos baseados em tipos de eventos.

## Fluxo de Publicação e Consumo de Eventos




## Detalhamento do Produtor e Consumidor

### 1. Produtor (Producer - MotorcycleService)

Ao receber uma requisição de cadastro de moto válida, o serviço de aplicação persiste o registro e dispara o evento assíncrono:

```
await _messageBus.PublishAsync(
    new MotorcycleRegisteredEvent(motorcycle.Id, motorcycle.Year, motorcycle.Model, motorcycle.Plate),
    cancellationToken);
```

### 2. Consumidor (Consumer - MotorcycleCreatedConsumer)

O consumidor escuta a fila dedicada e implementa a regra de negócio do desafio técnico: quando o ano de fabricação da moto for **2024**, uma notificação especial deve ser gerada e persistida:

```
public async Task Consume(ConsumeContext<MotorcycleRegisteredEvent> context)
{
    var message = context.Message;
    if (message.Year == 2024)
    {
        var notification = new MotorcycleNotification(
            id: Guid.NewGuid().ToString(),
            motorcycleId: message.MotorcycleId,
            year: message.Year,
            model: message.Model,
            plate: message.Plate,
            message: $"Moto ano 2024 cadastrada com sucesso com a placa {message.Plate}."
        );
        await _notificationRepository.AddAsync(notification, context.CancellationToken);
        await _unitOfWork.CommitAsync(context.CancellationToken);
    }
}
```

> **Auditoria no Painel do RabbitMQ:** O administrador pode acompanhar as filas ativas, taxas de entrega e mensagens em trânsito através do painel oficial em http://localhost:15672 (usuário: `guest`, senha: `guest`).

# 4. Especificação das Funcionalidades de Negócio

## 4.1. Gestão de Frotas de Motos

- **Cadastro de Moto:** Requer `identificador` único, `ano`, `modelo` e `placa` única. Caso a placa ou identificador já existam, a API retorna erro 400.
- **Consulta e Filtro por Placa:** Permite consultar todas as motos ou filtrar via query string `GET /motos?placa=XYZ`.
- **Atualização de Placa:** Endpoint `PUT /motos/{id}/placa` permite modificar exclusivamente a placa, garantindo que a nova placa não colida com outra moto existente.
- **Exclusão de Moto:** Endpoint `DELETE /motos/{id}`. Valida integridade referencial: caso a moto possua qualquer registro histórico de locação vinculado, a exclusão é rejeitada com erro 400.

## 4.2. Gestão de Motoristas Entregadores

- **Cadastro com Unicidade:** Validação obrigatória de unicidade de `CNPJ` e `numero_cnh` em nível de aplicação e banco de dados (índices únicos).
- **Categorias de CNH Aceitas:** Suporta categorias `A`, `B` e `A+B`.
- **Upload de Imagem da CNH:** Endpoint `POST /entregadores/{id}/cnh` recebe a imagem em formato `multipart/form-data`. O serviço valida formatos permitidos (`.png` e `.bmp`) e salva o arquivo em diretório seguro configurado, atualizando a URL no perfil do entregador.

## 4.3. Gestão de Locação de Motos e Regras Financeiras

O módulo de locações implementa as regras rigorosas de elegibilidade e tarifas contratuais:

> **Regra de Habilitação:** Apenas entregadores habilitados na **categoria A** (ou **A+B**) têm permissão para alugar motos. Entregadores que possuem apenas categoria B são bloqueados.

### Tabela de Planos Contratuais e Custos

| Plano (Duração) | Valor da Diária | Custo Base Contratual | Regra de Devolução Antecipada (Multa) |
|---|---|---|---|
| **7 dias** | R$ 30,00 / dia | R$ 210,00 | Diárias utilizadas + **20% de multa** sobre o valor das diárias não efetivadas |
| **15 dias** | R$ 28,00 / dia | R$ 420,00 | Diárias utilizadas + **40% de multa** sobre o valor das diárias não efetivadas |
| **30 dias** | R$ 22,00 / dia | R$ 660,00 | Valor contratual integral das diárias utilizadas |
| **45 dias** | R$ 20,00 / dia | R$ 900,00 | Valor contratual integral das diárias utilizadas |
| **50 dias** | R$ 18,00 / dia | R$ 900,00 | Valor contratual integral das diárias utilizadas |

**Regra de Atraso na Devolução:** Caso a moto seja devolvida após a data prevista de término, é cobrado o valor total base do plano contratado acrescido de uma **tarifa fixa de R$ 50,00 por diária adicional** excedente.

**Início Obrigatório da Locação:** A data de início da locação deve ser obrigatoriamente o **primeiro dia útil seguinte à data de criação** ( `D + 1` ).

## 4.4. Gestão de Pedidos e Entregas

- **Criação de Pedidos:** Pedidos com descrição e identificador são disponibilizados na plataforma com status `Available`.
- **Vinculação e Aceite:** Um entregador só pode aceitar uma entrega se tiver uma **locação ativa** no momento. O status passa para `Accepted`.
- **Conclusão da Entrega:** Apenas o motorista vinculado ao pedido tem permissão para marcar a entrega como `Completed`.

# 5. Painel Web Administrativo (ASP.NET Core MVC)

O projeto **DesafioBackend.Web** foi implementado em ASP.NET Core MVC para fornecer aos operadores e administradores uma interface moderna, rápida e intuitiva para controle total das operações da plataforma em `http://localhost:5001`.

## Principais Módulos da Interface Gráfica

| Módulo | Funcionalidade | Recursos em Destaque |
|---|---|---|
| **Autenticação & Sessão** | Login seguro para administradores ( `/Auth/Login` ). | Sessão por cookies seguros (HttpOnly), proteção contra CSRF (ValidateAntiForgeryToken) e repasse automático de token JWT para a API. |
| **Painel Geral (Admin)** | Visão panorâmica em tempo real com cards de métricas. | Totalizadores de motos, motoristas cadastrados, locações ativas e pedidos de entrega em andamento. |
| **Gestão de Motos** | Cadastro e consulta de veículos com botão de retorno. | Filtro dinâmico por placa, modal de cadastro e tabela de dados com formatação elegante. |
| **Gestão de Entregadores** | Controle de motoristas e habilitações. | Visualização do status da CNH, preview de imagem enviada e identificação rápida de categoria A/B/AB. |
| **Gestão de Locações** | Contratação de planos e baixa de devolução. | Cálculo automático de diárias, simulação de multas por devolução antecipada ou juros por atraso. |
| **Entregas Disponíveis** | Painel para vincular pedidos a motoristas elegíveis. | Lista suspensa inteligente: exibe apenas entregadores com moto atualmente alugada para evitar vínculos inválidos. |
| **Entregas em Andamento** | Acompanhamento de pedidos em trânsito. | Exibe motorista responsável, data/hora de aceite e botão de um clique para confirmação de entrega concluída. |
| **Entregas Concluídas** | Histórico e auditoria de entregas finalizadas. | Tabela formatada no padrão oficial: *Identificador* \| *Pedido* \| *Descrição* \| *Entregador* \| *Aceita em* \| *Concluído em*. |

# 6. Segurança, Autenticação e Documentação OpenAPI / Swagger

## Mecanismo de Autenticação JWT Bearer

A API implementa autenticação baseada em tokens **JSON Web Token (JWT)** assinados com algoritmo HMAC-SHA256.

- **Endpoint de Login:** `POST /auth/login` recebe usuário e senha (padrão: `admin` / `admin`) e retorna o token com tempo de expiração e claims de permissão.
- **Proteção de Endpoints:** As rotas de negócio exigem o cabeçalho `Authorization: Bearer <token>`, garantindo que requisições não autorizadas sejam rejeitadas com status **401 Unauthorized**.

## Configuração do Swagger com Suporte a Bearer Token

A interface do Swagger em `http://localhost:5000` está equipada com o recurso `AddSecurityDefinition("Bearer", ...)`:

1. Ao acessar a documentação interativa, clique no botão **Authorize** (ícone de cadeado verde no topo).
2. Insira o token obtido no endpoint de autenticação no formato: `Bearer <seu_token>`.
3. A partir deste momento, todas as requisições disparadas pelo Swagger incluirão automaticamente o token de autorização.

## 7. Testes Automatizados e Confiabilidade

O projeto adota a pirâmide de testes para garantir que as regras financeiras e de integridade do domínio nunca sofram regressões.

| Projeto de Testes | Tipo de Teste | Quantidade | Status | Cenários Cobertos |
|---|---|---|---|---|
| **DesafioBackend.UnitTests** | Testes Unitários de Domínio e Casos de Uso | **21 testes** | 100% Aprovado | • Cálculo de tarifas para planos de 7, 15, 30, 45 e 50 dias<br>• Aplicação exata de multa de 20% (7 dias) e 40% (15 dias)<br>• Cálculo de taxa de R$ 50/dia por atraso na devolução<br>• Validação de CNH categoria A vs categoria B<br>• Invariantes de data de início (D+1) e integridade de entidades |
| **DesafioBackend.IntegrationTests** | Testes de Integração de Endpoints da API | **4 testes** | 100% Aprovado | • Fluxo de login e geração de JWT<br>• Cadastro e consulta de motos via HTTP<br>• Rejeição de duplicidade de placa e identificador<br>• Respostas padronizadas de erro 400 e 404 |

# 8. Guia Operacional, Credenciais e Depuração no Visual Studio

## Tabela Resumo de Credenciais e Acesso

| Recurso | Endereço / Host | Porta | Usuário | Senha |
|---|---|---|---|---|
| **Painel Web MVC** | `http://localhost:5001` | 5001 | `admin` | `admin` |
| **API Swagger** | `http://localhost:5000` | 5000 | `admin` | `admin` |
| **RabbitMQ Management** | `http://localhost:15672` | 15672 | `guest` | `guest` |
| **PostgreSQL** | `localhost` (banco: `desafiodb` ) | 5433 | `postgres` | `postgrespassword` |

## Procedimentos de Depuração no Visual Studio 2022

### Opção A: Depuração Local com Dependências no Docker (Recomendada)

1. Inicie apenas o PostgreSQL e o RabbitMQ em background:

```
docker compose up -d db rabbitmq
```

2. Abra a solução `DesafioBackend.sln` no Visual Studio 2022.
3. Configure inicialização múltipla: clique com botão direito na solução → **Configurar Projetos de Inicialização...** → Marque **Iniciar** para `DesafioBackend.Api` e `DesafioBackend.Web`.
4. Pressione **F5** para iniciar com breakpoints e Hot Reload ativos.

### Opção B: Anexar o Depurador ao Processo dentro do Container Docker

1. Suba a stack inteira no terminal: `docker compose up -d`
2. No Visual Studio, pressione `Ctrl + Alt + P` (**Depurar** → **Anexar ao Processo...**).
3. Em **Tipo de Conexão**, escolha **Docker (Contêiner do Linux)**.
4. Selecione o container `desafio_backend_api` ou `desafio_backend_web` e anexe ao processo `dotnet`.

**Relatório Técnico Concluído**
Sistema em conformidade técnica integral com todos os requisitos do desafio arquitetural.